



UNIVERSITAT
Carlemany

Formación europea
para desafíos reales

De:

 Planeta Formación y Universidades

Bloque 2- Bases de Datos y Modelado Relacional

Aplicaciones basadas en tecnologías web

Bloque 2 — Bases de Datos y Modelado Relacional

- Introducción a las bases de datos
- Tipos de bases de datos
- Introducción al modelo relacional
- Claves primarias (Primary Key)
- Claves foráneas (Foreign Key)
- Relaciones complejas
- Calidad del modelo y normalización
- Introducción al lenguaje SQL
- Filtrado y proyección de datos
- Combinación de tablas
- Agregación y análisis

Introducción a las bases de datos

- Qué es una base de datos y qué problema resuelve
- Datos persistentes frente a datos en memoria
- Sistemas de bases de datos y motores de almacenamiento

Qué es una base de datos y qué problema resuelve

Una **base de datos** es un sistema diseñado para **almacenar, organizar y recuperar información de forma persistente y fiable**.

Problemas que resuelve frente a otras alternativas:

-  Archivos sueltos (CSV, Excel): duplicación, errores, sin control
-  Memoria (listas, arrays): los datos se pierden al apagar el sistema
-  Código duro: difícil de consultar, evolucionar y mantener

Una base de datos permite:

-  persistencia de datos en el tiempo
-  reutilización de la información por distintos procesos
-  acceso concurrente por múltiples usuarios o sistemas
-  estructura y reglas sobre los datos (no todo vale)

 *Una base de datos no es solo “guardar datos”, es **poner orden y reglas** a la información.*

Datos persistentes frente a datos en memoria

Datos en memoria:

- viven solo mientras el programa está en ejecución
- son rápidos de acceder
- desaparecen al reiniciar el sistema

Ejemplo típico:

- listas, diccionarios, objetos en RAM

Datos persistentes:

- se almacenan en disco
- sobreviven a reinicios y fallos
- pueden ser compartidos por varios procesos

Ejemplo típico:

- bases de datos
- sistemas de ficheros estructurados

Comparación clave:

- memoria → rendimiento inmediato
- persistencia → continuidad y trazabilidad

 *Los sistemas reales combinan ambos: lógica en memoria, estado crítico persistente.*

Sistemas de bases de datos y motores de almacenamiento

Un **sistema de bases de datos** no es solo “una base de datos”, sino un conjunto de componentes:

-  **Modelo de datos:** cómo se representan los datos (tablas, documentos, grafos)
-  **Motor:** cómo se almacenan y recuperan físicamente
-  **Lenguaje de consulta:** cómo se accede a la información
-  **Reglas:** integridad, consistencia, restricciones

Un **motor de base de datos** se encarga de:

- gestionar el almacenamiento en disco
- optimizar consultas
- controlar concurrencia
- garantizar consistencia de los datos

Ejemplos de motores (conceptual):

- motores relacionales
- motores documentales
- motores de grafos

 *Elegir una base de datos es elegir **cómo piensas consultar y relacionar los datos**, no solo dónde guardarlos.*

Tipos de bases de datos

- Bases de datos relacionales: concepto y características
- Bases de datos NoSQL: motivación y limitaciones
- Bases de datos documentales (ej. JSON, documentos)
- Bases de datos clave–valor
- Bases de datos orientadas a grafos
- Cuándo usar una base de datos relacional frente a NoSQL

Bases de datos relacionales: concepto y características

Las **bases de datos relacionales** organizan la información en **tablas** compuestas por:

- filas (registros)
- columnas (atributos)
- relaciones explícitas entre tablas

Características clave:

-  **estructura fija**: el esquema se define antes de insertar datos
-  **claves primarias y foráneas** para identificar y relacionar registros
-  **SQL** como lenguaje estándar de consulta
-  **integridad referencial** y reglas de consistencia

Ventajas:

- ideal para datos estructurados
- potente capacidad de consulta (JOIN, agregados)
- modelo claro y predecible

Limitaciones:

- menos flexible ante cambios frecuentes de esquema
- requiere diseño previo cuidadoso

 *Cuando los datos están bien estructurados y relacionados, el modelo relacional sigue siendo imbatible.*

Bases de datos NoSQL: motivación y limitaciones

El término **NoSQL** agrupa bases de datos que **no siguen estrictamente el modelo relacional**.

Motivaciones principales:

- manejar grandes volúmenes de datos
- esquemas flexibles o cambiantes
- escalabilidad horizontal sencilla

Características comunes:

-  no usan tablas relacionales clásicas
-  no suelen usar JOINS complejos
-  priorizan rendimiento y flexibilidad

Limitaciones importantes:

- menor capacidad para consultas complejas
- relaciones implícitas (gestionadas por la aplicación)
- consistencia a veces eventual

 *NoSQL no sustituye a SQL: resuelve problemas distintos.*

Bases de datos documentales

Las **bases de datos documentales** almacenan información como documentos completos (normalmente JSON).

Cada documento:

- contiene datos anidados
- puede tener estructura diferente a otros documentos
- representa una “unidad lógica” completa

Ventajas:

- gran flexibilidad de esquema
- lectura rápida de documentos completos
- ideal cuando los datos se consumen juntos

Inconvenientes:

- duplicación de datos frecuente
- consultas agregadas complejas
- difícil representar relaciones ricas

 *Funcionan bien cuando lees “el todo”, no cuando necesitas cruzar muchos datos.*

Bases de datos clave–valor

En una base de datos **clave–valor**:

- cada dato se accede mediante una clave única
- el valor es opaco para el sistema

Modelo:

clave → valor

Características:

- extremadamente rápidas
- muy simples conceptualmente
- sin relaciones entre datos

Casos típicos:

- caché
- sesiones de usuario
- configuraciones

Limitaciones:

- no permiten consultas complejas
- no existe estructura interna explotable

 *Son excelentes para acceso directo, inútiles para análisis relacional.*



Bases de datos orientadas a grafos

Las bases de datos de **grafos** modelan los datos como:

- nodos (entidades)
- aristas (relaciones)
- propiedades asociadas a ambos

Fortalezas:

- relaciones complejas y profundas
- recorridos y dependencias
- consultas del tipo “camino más corto”

Ejemplos de uso:

- redes sociales
- recomendaciones
- dependencias complejas

Debilidades:

- menos eficientes para agregados masivos
- curva de aprendizaje mayor



Cuando las relaciones son el dato principal, el grafo es natural.

Cuándo usar una base de datos relacional frente a NoSQL

Elegir tipo de base de datos depende de:

- estructura de los datos
- tipo de consultas necesarias
- evolución esperada del sistema

Relacional es preferible cuando:

- hay relaciones claras entre entidades
- se necesitan JOINS y agregados
- la consistencia es crítica

NoSQL es preferible cuando:

- el esquema cambia constantemente
- se prioriza escalabilidad horizontal
- las relaciones son simples o inexistentes

 *No existe “la mejor base de datos”, existe la **más adecuada al problema.***

Introducción al modelo relacional

- El modelo relacional: tablas, filas y columnas
- Relaciones entre tablas como concepto central
- Identidad de los datos en el modelo relacional

El modelo relacional: tablas, filas y columnas

El **modelo relacional** organiza la información en **tablas** (relations), formadas por:

- **filas** → instancias concretas de información
- **columnas** → atributos con un significado definido

Cada tabla representa un **concepto del dominio** (una entidad lógica), no una pantalla ni un proceso.

Ejemplo conceptual:

- una tabla describe *qué cosas existen*
- no *qué se hace con ellas*

Características clave:

- todas las filas siguen la misma estructura
- cada columna tiene un tipo y un significado claro
- el orden de las filas **no importa**

 *Una tabla bien diseñada representa una idea estable del dominio, no una necesidad puntual.*

Relaciones entre tablas como concepto central

En el modelo relacional, **las relaciones son tan importantes como los datos.**

Las tablas **no viven aisladas**:

- una tabla suele depender de otra
- los datos adquieren significado al combinarse

Ejemplos conceptuales de relaciones:

- un elemento pertenece a un tipo
- un registro referencia a otro
- un evento afecta a múltiples entidades

Las relaciones permiten:

- evitar duplicación de datos
- mantener consistencia
- responder preguntas complejas

 *Si tu modelo no tiene relaciones, probablemente no es relacional de verdad.*

Identidad de los datos en el modelo relacional

En una base de datos relacional, **cada fila debe ser identificable de forma única.**

La identidad:

- no depende de la posición
- no depende del orden
- no depende del momento de inserción

Se consigue mediante:

- una **clave primaria**
- estable en el tiempo
- única y no nula

Problemas sin identidad clara:

- registros duplicados
- referencias ambiguas
- imposibilidad de relacionar datos con seguridad

 *Sin identidad, no hay relaciones fiables ni modelo consistente.*

Claves primarias (Primary Key)

- Qué es una Primary Key y por qué es obligatoria
- Propiedades formales de una Primary Key
- Claves naturales: ventajas y riesgos
- Claves artificiales: uso mayoritario en la práctica

Qué es una Primary Key y por qué es obligatoria

Una **Primary Key (PK)** es un atributo (o conjunto de atributos) que **identifica de forma única** cada fila de una tabla.

Propósito fundamental:

- distinguir cada registro de todos los demás
- permitir referencias desde otras tablas
- garantizar integridad del modelo

Sin Primary Key:

- no se puede asegurar unicidad
- las relaciones se vuelven ambiguas
- los JOINS dejan de ser fiables

En bases de datos relacionales:

- **toda tabla debe tener una PK**
- incluso tablas intermedias o de relación

 *La Primary Key no es opcional: es la base de todo el modelo.*

Propiedades formales de una Primary Key

Una Primary Key debe cumplir siempre:

- **Unicidad** → no puede repetirse
- **No nulidad** → nunca puede ser NULL
- **Estabilidad** → no debería cambiar en el tiempo

Buenas prácticas:

- evitar claves que puedan cambiar
- evitar significados “humanos” frágiles
- preferir valores simples y técnicos

Ejemplos de malas PK:

- nombres
- códigos externos no controlados
- combinaciones largas sin necesidad

 *Una PK correcta es aburrida... y eso es bueno.*



Claves naturales: ventajas y riesgos

Una **clave natural** es un atributo del dominio que parece identificar de forma única:

- DNI
- email
- matrícula
- código externo

Ventajas:

- semántica clara
- no requiere campo adicional

Riesgos habituales:

- pueden cambiar
- pueden no ser tan únicas como parecen
- pueden tener reglas externas

Consecuencia:

- cambios costosos en cascada
- relaciones frágiles



Lo que hoy parece estable, mañana puede no serlo.



Claves artificiales: uso mayoritario en la práctica

Una **clave artificial** es un identificador creado exclusivamente para la base de datos:

- entero autoincremental
- UUID / GUID

Ventajas:

- estabilidad total
- simplicidad en relaciones
- independencia del dominio

Desventajas:

- no tiene significado semántico
- requiere combinar con otros campos para entender el dato

En sistemas reales:

- la mayoría de tablas usan claves artificiales
- las claves naturales se mantienen como atributos con restricciones



Separar identidad técnica y significado de negocio simplifica el diseño.

Claves foráneas (Foreign Key)

- Qué es una Foreign Key y cómo conecta tablas
- Integridad referencial: consistencia entre entidades
- Qué problemas aparecen cuando no se usan Foreign Keys
- Relaciones uno a muchos (1:N) en el modelo relacional

Qué es una Foreign Key y cómo conecta tablas

Una **Foreign Key (FK)** es un atributo (o conjunto de atributos) en una tabla que **referencia la Primary Key de otra tabla**.

Su función principal:

- conectar datos entre tablas
- expresar relaciones explícitas
- garantizar coherencia entre entidades

Conceptualmente:

- una fila “apunta” a otra
- la relación queda definida en el esquema

Ejemplo abstracto:

- una entidad “A” pertenece a una entidad “B”
- la FK vive en “A” y referencia la PK de “B”

 *Las Foreign Keys convierten tablas aisladas en un modelo relacional real.*

Integridad referencial: consistencia entre entidades

La **integridad referencial** es la garantía de que:

- una Foreign Key siempre apunta a una fila existente
- no hay referencias “rotas”

Gracias a la integridad referencial:

- no pueden existir relaciones inválidas
- el sistema evita errores de datos automáticamente

Ejemplos de incoherencia evitados:

- referencias a registros inexistentes
- datos huérfanos
- relaciones inconsistentes

 *La base de datos protege el modelo incluso cuando la aplicación falla.*

Qué problemas aparecen cuando no se usan Foreign Keys

Cuando no se definen Foreign Keys:

- la base de datos no valida relaciones
- la coherencia depende del código
- los errores aparecen con el tiempo

Problemas típicos:

- registros huérfanos
- datos duplicados
- JOINS que no devuelven lo esperado

Además:

- las consultas son menos fiables
- el mantenimiento se complica

 *Si la base de datos no conoce las relaciones, no puede protegerlas.*

Relaciones uno a muchos (1:N) en el modelo relacional

Una relación **uno a muchos (1:N)** significa:

- una fila de la tabla “uno”
- puede estar relacionada con muchas filas de la tabla “muchos”

Implementación:

- la **Foreign Key se coloca en la tabla “muchos”**
- apunta a la PK de la tabla “uno”

Características:

- modelo simple
- consultas claras
- integridad fácil de mantener

Ejemplos conceptuales:

- un tipo con muchos elementos
- una categoría con múltiples registros

 *La relación 1:N es la más común y la más fácil de modelar correctamente.*

Relaciones complejas

- Relaciones muchos a muchos (N:M) y sus implicaciones
- Tablas intermedias como solución a relaciones N:M
- Atributos en relaciones: cuándo la relación tiene datos propios

Relaciones muchos a muchos (N:M) y sus implicaciones

Una relación **muchos a muchos (N:M)** ocurre cuando:

- una fila de la tabla A puede relacionarse con muchas filas de B
- y una fila de B puede relacionarse con muchas filas de A

Ejemplos conceptuales:

- elementos que participan en múltiples eventos
- entidades que se ven afectadas por múltiples factores

Problema clave:

- **el modelo relacional NO permite N:M directas**
- no se pueden poner múltiples valores en una FK

 *Si intentas modelar una N:M directamente, el diseño ya está mal.*

Por qué una relación N:M no cabe en una sola tabla

Intentar representar una N:M en una sola tabla provoca:

- columnas repetidas
- valores múltiples en una celda
- violación de la Primera Forma Normal (1NF)

Ejemplos de malas soluciones:

- listas separadas por comas
- columnas duplicadas (Hazard1, Hazard2, Hazard3)
- estructuras difíciles de consultar

Consecuencias:

- consultas complejas o imposibles
- agregaciones incorrectas
- datos inconsistentes

 *El modelo relacional exige atomicidad: un dato por celda.*

Tablas intermedias como solución a relaciones N:M

La solución estándar a una relación N:M es una **tabla intermedia**.

Características:

- contiene al menos dos Foreign Keys
- cada FK apunta a una de las tablas originales
- cada fila representa **una relación concreta**

Ventajas:

- mantiene el modelo normalizado
- permite consultas claras
- escala correctamente

Conceptualmente:

- la relación se convierte en una entidad propia

 *Cuando aparece una tabla intermedia, el modelo gana expresividad.*

Atributos en relaciones: cuándo la relación tiene datos propios

A veces la relación **no es solo un enlace**, sino que tiene información asociada.

Ejemplos conceptuales:

- intensidad de una relación
- porcentaje, valor o nivel
- condiciones específicas

En estos casos:

- la tabla intermedia **no es opcional**
- es la única forma correcta de almacenar esos datos

Errores comunes:

- mover esos atributos a una de las tablas principales
- duplicar datos

 *Si un atributo depende de dos entidades, pertenece a la relación.*

Calidad del modelo y normalización

- Redundancia de datos y por qué debe evitarse
- Primera Forma Normal (1NF)
- Segunda Forma Normal (2NF)
- Tercera Forma Normal (3NF)
- Anomalías de inserción, actualización y borrado

Redundancia de datos y por qué debe evitarse

La **redundancia de datos** ocurre cuando la misma información:

- se almacena en varios lugares
- debe mantenerse sincronizada manualmente

Problemas que introduce:

- inconsistencias (valores distintos para lo mismo)
- mayor consumo de almacenamiento
- errores difíciles de detectar

Ejemplos conceptuales de redundancia:

- repetir nombres o descripciones en varias tablas
- copiar datos que ya pueden obtenerse mediante relaciones

 *La redundancia parece cómoda al principio, pero es costosa a largo plazo.*

Primera Forma Normal (1NF)

La **Primera Forma Normal (1NF)** establece que:

- cada celda debe contener un valor **atómico**
- no deben existir listas ni grupos repetidos
- todas las filas deben tener la misma estructura

Violaciones típicas de 1NF:

- valores separados por comas
- columnas duplicadas (campo1, campo2, campo3)
- estructuras variables dentro de una misma tabla

 *1NF garantiza que los datos puedan ser consultados correctamente.*

Segunda Forma Normal (2NF)

La **Segunda Forma Normal (2NF)** se aplica a tablas con:

- claves primarias compuestas

Requisito:

- todos los atributos no clave deben depender de **toda** la clave
- no solo de una parte

Problema que evita:

- datos que “pertenecen” solo a una parte de la clave
- duplicaciones ocultas

 *Si un dato no depende completamente de la clave, está mal ubicado.*

Tercera Forma Normal (3NF)

La **Tercera Forma Normal (3NF)** exige que:

- los atributos no clave no dependan de otros atributos no clave

Es decir:

- no debe haber dependencias transitivas
- cada atributo depende directamente de la clave

Problemas que evita:

- actualizaciones inconsistentes
- duplicación semántica
- errores silenciosos

 *3NF separa conceptos que no deberían mezclarse.*

Anomalías de inserción, actualización y borrado

Un modelo no normalizado genera **anomalías**:

- **Inserción**: no se puede añadir un dato sin otro relacionado
- **Actualización**: cambiar un valor exige modificar muchas filas
- **Borrado**: eliminar un dato provoca pérdida de información no deseada

Estas anomalías:

- no siempre se detectan al principio
- aparecen cuando el sistema crece

 *La normalización no es teoría académica: previene errores reales.*

Introducción al lenguaje SQL

- SQL como lenguaje declarativo
- La sentencia SELECT como base de las consultas
- SELECT y FROM: qué datos se devuelven
- WHERE: filtrado de filas

SQL como lenguaje declarativo

SQL es un lenguaje **declarativo**, no imperativo.

Esto significa que:

- describimos **qué datos queremos**
- no **cómo obtenerlos**

El motor de la base de datos:

- decide el plan de ejecución
- optimiza accesos e índices
- gestiona el rendimiento

Diferencia clave:

- imperativo → “haz esto paso a paso”
- declarativo → “quiero este resultado”

 *SQL se centra en el resultado, no en el algoritmo.*

La sentencia **SELECT** como base de las consultas

La sentencia **SELECT** es el núcleo de SQL.

Sirve para:

- recuperar datos
- analizar información
- construir métricas

Estructura conceptual mínima:

```
SELECT columnas  
FROM tabla;
```

Características:

- no modifica datos
- puede combinarse con muchas cláusulas
- siempre devuelve un conjunto de filas

 *Todo análisis en SQL empieza con **SELECT**.*

SELECT y FROM: qué datos se devuelven

- **SELECT** define **qué columnas** se muestran
- **FROM** define **de dónde salen los datos**

Ejemplo conceptual:

```
SELECT nombre, valor  
FROM elementos;
```

Buenas prácticas:

- evitar `SELECT *` en consultas reales
- seleccionar solo las columnas necesarias

Ventaja:

- consultas más claras
- mejor rendimiento

 *SELECT * es cómodo, pero rara vez es buena idea.*

WHERE: filtrado de filas

La cláusula **WHERE** filtra las filas **antes** de devolver resultados.

Permite:

- limitar el conjunto de datos
- reducir volumen
- expresar condiciones lógicas

Ejemplo conceptual:

```
SELECT *  
FROM elementos  
WHERE valor > 100;
```

Características:

- se evalúa fila a fila
- afecta al resultado final
- no se aplica a agregados

 *WHERE reduce datos antes de cualquier cálculo.*

Operadores lógicos y condiciones compuestas

En WHERE pueden combinarse condiciones usando:

- AND
- OR
- NOT

Ejemplo conceptual:

```
WHERE valor > 100  
AND tipo = 'A';
```

Errores comunes:

- olvidar paréntesis
- confundir precedencia lógica
- usar OR cuando se necesita AND

 *La lógica mal expresada devuelve resultados incorrectos.*

Combinación de tablas

- JOIN: concepto general
- INNER JOIN: combinación de datos relacionados
- LEFT JOIN: conservar filas sin correspondencia

JOIN: concepto general

Un **JOIN** permite **combinar filas de dos o más tablas** en función de una relación lógica entre ellas.

Conceptualmente:

- no “mezcla tablas”
- **empareja filas relacionadas**

La relación suele basarse en:

- una **Primary Key**
- y una **Foreign Key**

Ejemplo conceptual:

- Tabla A —(FK → PK)— Tabla B

 *JOIN es la materialización de una relación en una consulta.*

INNER JOIN: combinar solo datos relacionados

INNER JOIN devuelve solo las filas que:

- tienen correspondencia en **ambas tablas**

Ejemplo conceptual:

```
SELECT *  
FROM A  
INNER JOIN B ON A.id = B.a_id;
```

Características:

- filas sin relación se descartan
- resultado más “estricto”
- ideal cuando la relación es obligatoria

Consecuencia:

- si faltan datos relacionados, **no aparecen**

 *INNER JOIN responde a “dame solo lo que está correctamente relacionado”.*

LEFT JOIN: conservar filas sin correspondencia

LEFT JOIN devuelve:

- todas las filas de la tabla izquierda
- aunque no exista relación en la tabla derecha

Ejemplo conceptual:

```
SELECT *  
FROM A  
LEFT JOIN B ON A.id = B.a_id;
```

Resultado:

- columnas de B serán NULL si no hay relación

Cuándo usarlo:

- cuando la relación es opcional
- cuando quieres ver “todo”, exista o no relación

 *LEFT JOIN permite detectar ausencias, no solo presencias.*

Elegir el JOIN según la pregunta

La elección del JOIN **depende de la pregunta**, no del modelo.

Preguntas típicas:

- “solo los que tienen relación” → INNER JOIN
- “todos, aunque no tengan relación” → LEFT JOIN

Error común:

- usar INNER JOIN por defecto
- perder filas sin darse cuenta

 *Un JOIN incorrecto da resultados incorrectos, aunque la query “funcione”.*

Agregación y análisis

- Funciones agregadas: SUM, COUNT, AVG, MIN, MAX
- GROUP BY: agrupar resultados
- HAVING: filtrar datos agregados



Funciones agregadas: SUM, COUNT, AVG, MIN, MAX

Las **funciones agregadas** permiten calcular valores **a partir de múltiples filas**.

Funciones más comunes:

- COUNT → número de filas
- SUM → suma de valores
- AVG → media
- MIN / MAX → valores extremos

Ejemplo conceptual:

```
SELECT SUM(valor)  
FROM registros;
```

Características:

- operan sobre conjuntos de filas
- devuelven un único valor por grupo
- ignoran valores NULL (en general)



Las funciones agregadas transforman datos en métricas.



Funciones agregadas: SUM, COUNT, AVG, MIN, MAX

Las **funciones agregadas** permiten calcular valores **a partir de múltiples filas**.

Funciones más comunes:

- COUNT → número de filas
- SUM → suma de valores
- AVG → media
- MIN / MAX → valores extremos

Ejemplo conceptual:

```
SELECT SUM(valor)  
FROM registros;
```

Características:

- operan sobre conjuntos de filas
- devuelven un único valor por grupo
- ignoran valores NULL (en general)



Las funciones agregadas transforman datos en métricas.



GROUP BY: agrupar datos para análisis

GROUP BY permite dividir las filas en **grupos** y aplicar agregaciones a cada uno.

Ejemplo conceptual:

```
SELECT tipo, SUM(valor)
FROM registros
GROUP BY tipo;
```

Resultado:

- una fila por grupo
- una métrica por grupo

Regla fundamental:

- toda columna en SELECT que no sea agregada **debe aparecer en GROUP BY**



GROUP BY define “sobre qué” se calculan las métricas.



Diferencia entre WHERE y HAVING

Aunque se parecen, **WHERE** y **HAVING** no hacen lo mismo.

- **WHERE** filtra **filas individuales**
- **HAVING** filtra **grupos agregados**

Ejemplo conceptual:

```
SELECT tipo, SUM(valor)
FROM registros
GROUP BY tipo
HAVING SUM(valor) > 1000;
```

Error común:

- usar WHERE con funciones agregadas
- filtrar demasiado pronto o demasiado tarde



WHERE filtra antes de agrupar, HAVING después.



Orden lógico de ejecución de una query SQL

Aunque se escriba en este orden:

- SELECT ...
- FROM ...
- WHERE ...
- GROUP BY ...
- HAVING ...

SQL se ejecuta conceptualmente así:

- 1 FROM
- 2 WHERE
- 3 GROUP BY
- 4 HAVING
- 5 SELECT

Comprender este orden explica:

- por qué WHERE no ve agregados
- por qué HAVING sí



Entender el orden lógico evita errores sutiles.



Consultas agregadas con JOINS

Las métricas reales suelen requerir:

- JOINS para unir tablas
- GROUP BY para agrupar
- funciones agregadas para calcular

Ejemplo:

```
SELECT categoria, SUM(valor)
FROM A
JOIN B ON ...
GROUP BY categoria;
```

Buenas prácticas:

- JOIN antes de agrupar
- agrupar por atributos estables
- validar resultados con datasets pequeños

 *La potencia del modelo relacional se demuestra en estas consultas.*



Errores comunes en consultas agregadas

Errores frecuentes:

- olvidar GROUP BY
- agrupar por demasiadas columnas
- usar INNER JOIN cuando faltan relaciones
- confundir WHERE y HAVING

Consecuencias:

- resultados incorrectos
- métricas infladas o incompletas
- decisiones erróneas basadas en datos mal agregados



Una query “válida” no siempre es una query “correcta”.

Pensar en métricas al diseñar el modelo

Un buen modelo relacional:

- facilita consultas agregadas
- reduce complejidad de SQL
- evita cálculos innecesarios

Diseñar pensando en:

- qué se va a sumar
- por qué se va a agrupar
- qué relaciones serán necesarias

 *El diseño del modelo y las queries no son independientes.*